

EXPERIMENT NO: 2

Name of the student: Adithya Menon

Roll No: 15

Division: B

Batch: A

Aim of the Experiment: Write a program to implement Diffie-Hellman Algorithm.

Learning Objectives: To understand the principles of symmetric key cryptography.

To understand the Diffie-Hellman Key exchange algorithm.

To understand the possible attacks on Diffie-Hellman.

Lab Outcomes: Apply the cryptosystem to ensure secure key exchange between sender and receiver.

Tools & Software Required: Python

Theory:

(Explain following terms with one example)

- **Diffie-Hellman key Exchange Algorithm:**

The Diffie-Hellman key exchange (often abbreviated DH) is a cryptographic protocol that allows two parties (commonly called Alice and Bob) to agree on a shared secret key over a completely public (insecure) communication channel — without ever sending the actual secret key itself.

This shared secret can then be used as the key for a symmetric encryption algorithm (e.g., AES) to securely encrypt messages.

Core Mathematical Idea

The security relies on modular exponentiation being easy to compute in one direction but extremely hard to reverse (the discrete logarithm problem).

**Key property used: $(g^b)^a \bmod p = (g^a)^b \bmod p = g^{(a \cdot b)} \bmod p$
(Exponentiation is commutative in this modular setting.)**

- **Example:**

Let's use very small numbers for clarity (real-world use uses huge primes, 2048+ bits).

Public values (everyone knows these — Alice, Bob, and any eavesdropper):

- **Prime number $p = 23$**
- **Base (generator) $g = 5$**

Step 1: Alice chooses her private key Alice picks a secret number $a = 6$ (kept secret).

Step 2: Alice computes her public value $A = g^a \bmod p = 5^6 \bmod 23$
 $5^1 = 5$
 $5^2 = 25 \bmod 23 = 2$
 $5^3 = 5 \cdot 2 = 10$
 $5^4 = 5 \cdot 10 = 50 \bmod 23 = 4$
 $5^5 = 5 \cdot 4 = 20$
 $5^6 = 5 \cdot 20 = 100 \bmod 23 = 8$

Alice → sends $A = 8$ to Bob (public).

Step 3: Bob chooses his private key Bob picks a secret number $b = 15$ (kept secret).

Step 4: Bob computes his public value $B = g^b \bmod p = 5^{15} \bmod 23$

(We can compute this efficiently using successive squaring): $5^{15} \bmod 23 = 19$

Bob → sends $B = 19$ to Alice (public).

Step 5: Alice computes the shared secret $KA = B^a \bmod p = 19^6 \bmod 23$

$19^2 \bmod 23 = 361 \bmod 23 = 361 - 15 \cdot 23 = 361 - 345 = 16$
 $19^4 = 16^2 = 256$
 $\bmod 23 = 256 - 11 \cdot 23 = 256 - 253 = 3$
 $19^6 = 19^4 \cdot 19^2 = 3 \cdot 16 = 48 \bmod 23 = 2$

Alice gets shared secret = 2

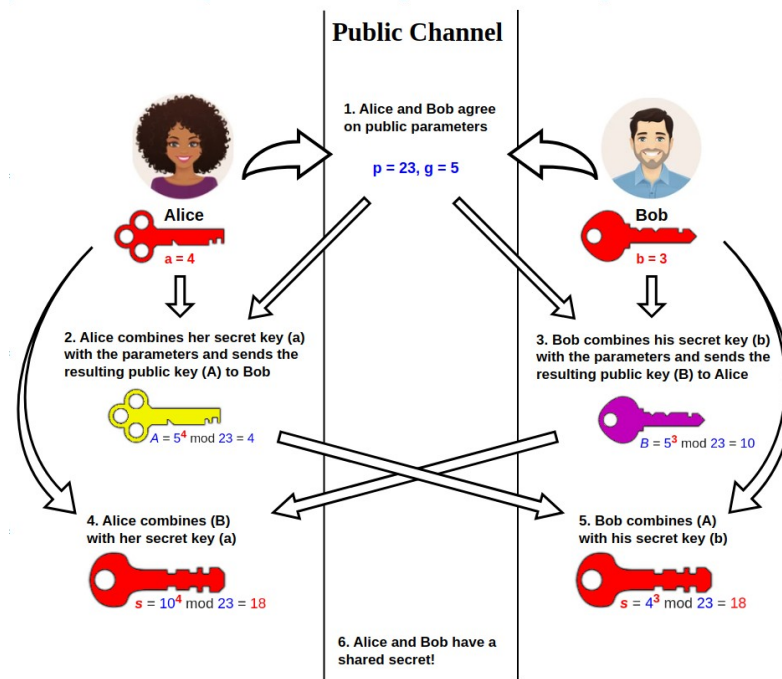
Step 6: Bob computes the shared secret $KB = A^b \bmod p = 8^{15} \bmod 23$

(Again using efficient exponentiation): $8^{15} \bmod 23 = 2$

Bob also gets shared secret = 2

Result Both now have the same secret number 2 — even though they never sent it directly. An eavesdropper sees only: $p=23$, $g=5$, $A=8$, $B=19$ Finding the secret requires solving the discrete logarithm (very hard with large p).

-



- **Primitive Root:**

Algorithm/Procedure:

1. Start
2. Input prime number p
3. Input primitive root g
4. Check smaller prime number is primitive root of higher prime number p . (A number g is a primitive root modulo p if:
 $g^1, g^2, g^3, \dots, g^{p-1} \pmod{p}$ produces all integers from: 1, 2, 3, ..., $p-1$ in some order.)
5. User A chooses private key a
6. User B chooses private key b
7. Compute public key of A: $A = g^a \mod p$
8. Compute public key of B: $B = g^b \mod p$
9. Exchange public keys
10. Compute secret key at A: $K_A = B^a \mod p$
11. Compute secret key at B: $K_B = A^b \mod p$
12. If $K_A = K_B \rightarrow$ Key exchange successful

13. Stop

Source Code:

Diffie-Hellman :

```
def is_primitive_root(g, p):
```

```
    if pow(g, p-1, p) != 1:
```

```
        return False
```

```
    factors = set()
```

```
    n = p - 1
```

```
    i = 2
```

```
    while i * i <= n:
```

```
        if n % i == 0:
```

```
            factors.add(i)
```

```
            while n % i == 0:
```

```
                n //= i
```

```
            i += 1
```

```
    if n > 1:
```

```
        factors.add(n)
```

```
    for q in factors:
```

```
        if pow(g, (p-1)//q, p) == 1:
```

```
            return False
```

```
    return True
```

```
p = int(input("Enter a prime number p: "))
```

```
g = int(input("Enter primitive root g modulo p: "))
```

```
while not is_primitive_root(g, p):
```

```
    print(f"{g} is NOT a primitive root modulo {p}")
```

```
    g = int(input("Enter a valid primitive root g: "))
```

```
print(f"✓ {g} is a primitive root modulo {p}")
```

```
a = int(input("User A - enter your private key a: "))
```

```
b = int(input("User B - enter your private key b: "))
```

```
A = pow(g, a, p)
```

```
B = pow(g, b, p)
```

```
print("\nPublic keys exchanged:")
```

```
print(f"Public key of A: {A}")
```

```
print(f"Public key of B: {B}")
```

```
KA = pow(B, a, p)
```

```
KB = pow(A, b, p)
```

```
print("\nComputed shared secrets:")
```

```
print(f"KA (at A) = {KA}")
```

```
print(f"KB (at B) = {KB}")
```

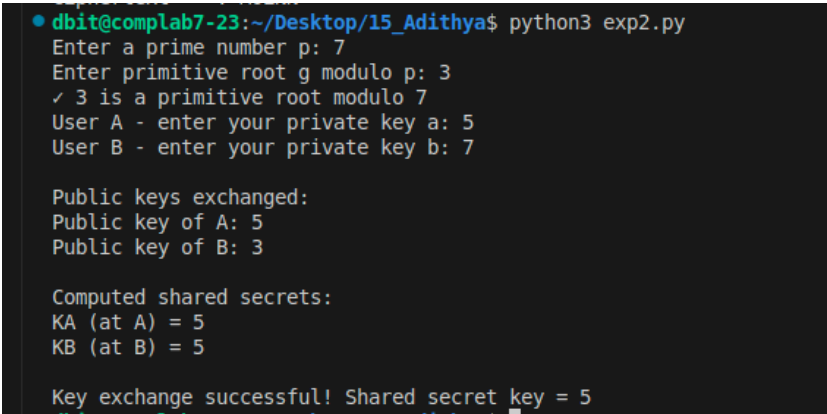
```
if KA == KB:
```

```
    print("\nKey exchange successful! Shared secret key =", KA)
```

```
else:
```

```
    print("\nKey exchange FAILED!")
```

Output Screenshots:



```
dbit@complab7-23:~/Desktop/15_Adithya$ python3 exp2.py
Enter a prime number p: 7
Enter primitive root g modulo p: 3
✓ 3 is a primitive root modulo 7
User A - enter your private key a: 5
User B - enter your private key b: 7

Public keys exchanged:
Public key of A: 5
Public key of B: 3

Computed shared secrets:
KA (at A) = 5
KB (at B) = 5

Key exchange successful! Shared secret key = 5
```

Conclusion :

In conclusion, the Diffie-Hellman key exchange stands as one of the most groundbreaking achievements in the history of cryptography. Introduced in 1976 by Whitfield Diffie and Martin Hellman, it solved a fundamental problem that had long challenged secure communication: how two parties can establish a shared secret key over an completely insecure public channel—without ever transmitting the key itself.

By cleverly leveraging modular exponentiation and the computational hardness of the discrete logarithm problem, Diffie-Hellman enables Alice and Bob to independently compute the identical secret value ($K_A = K_B$), even while an eavesdropper observes every public message exchanged. This elegant mechanism laid the foundation for modern public-key cryptography and made secure symmetric encryption practical over open networks.

Key strengths of Diffie-Hellman include:

- **No need for prior shared secrets or secure out-of-band channels**
- **Provision of perfect forward secrecy (when used in its ephemeral form — DHE / ECDHE) — past session keys remain protected even if long-term keys are later compromised**
- **Widespread use as a building block in nearly every major security protocol today (TLS, SSH, IPsec, Signal, VPNs, etc.)**